

Architektura a klientská implementace audio-reaktivních videoklipů v PWA aplikacích s využitím WebGPU a WebCodecs

Tvorba vizuálního obsahu pro platformu YouTube na základě audio stop generovaných systémy umělé inteligence, jako je Suno AI, představuje v klientském prostředí specifický inženýrský problém. YouTube ze své podstaty nepodporuje nahrávání čistých audio souborů, což tvůrce nutí k jejich konverzi na video formáty. Tradiční řešení se opírala buď o cloudové vykreslovací služby, nebo o lokální softwarovou emulaci bez hardwarové akcelerace, což vedlo k vysokým finančním nákladům nebo k extrémní časové a energetické náročnosti na straně uživatele.¹ S technologickým stavem v roce 2026, kdy standard WebGPU dosáhl široké nativní podpory napříč všemi hlavními internetovými prohlížeči, se však otevírá možnost realizovat vysoce výkonnou, nelineární generativní video pipeline přímo v klientské aplikaci typu Progressive Web App (PWA).² Tento report poskytuje hloubkovou analýzu architektury, která propojuje offline audio analýzu s hardwarově akcelerovaným renderováním pomocí WebGPU a nízkoúrovňovým kódováním videa prostřednictvím WebCodecs API.¹

Analýza technologických možností a architektonických přístupů

Při návrhu PWA aplikace pro klientský export videa je nutné zvolit optimální technologii pro zachytávání, kódování a ukládání vyrenderovaných snímků.² Historicky se vývojáři museli spoléhat na softwarové kompilace knihoven typu FFmpeg do WebAssembly (WASM) nebo na nahrávání plátna v reálném čase.³

Srovnání klientských exportních metod

Následující tabulka srovnává klíčové vlastnosti, limity a výkonnostní parametry čtyř hlavních architektonických přístupů dostupných v moderních webových prohlížečích.

Parametr / Vlastnost	WebGPU + WebCodecs (např. Mediabunny)	MediaRecorder API	FFmpeg.wasm (WebAssembly / CPU)	Softwarové JS/WASM kodéry (např. H264MP4Encoder)
Rychlost	Extrémní (hardwarově)	Omezena na reálný čas	Velmi pomalá (softwarová)	Střední až pomalá ³

exportu	akcelerovaná, až 10–20x rychlejší než reálný čas) ¹	(stejná jako délka přehrávaného audia) ³	emulace na CPU bez GPU akcelerace) ³	
Kontrola nad snímky	Absolutní (snímkově přesná kontrola bez rizika vynechání snímků) ³	Žádná (závislá na vykreslovací smyčce prohlížeče, hrozí vynechávání snímků při záseku UI) ³	Absolutní (sekvenční zápis snímků) ⁸	Absolutní ⁸
Limity velikosti souboru	Bez limitu (při průběžném zápisu přes File System Access API nebo OPFS) ²	Omezeno operační paměť RAM (při ukládání výsledného souboru do Blob)	Kritické (WASM paměťový limit, selhává u souborů nad 100 MB) ³	Hardwarový limit alokace WASM paměti (strop na úrovni cca 4 GB) ⁸
Podpora kontejnerů	MP4, WebM, MKV, MOV ²	Primárně WebM (Chrome), MP4/M4V (Safari) ⁷	Jakýkoliv kontejner podporovaný kompilací FFmpeg ¹¹	Výhradně MP4 ⁸
Režie procesoru (CPU)	Velmi nízká (cca 8 %, procesor plní pouze roli orchestrátoru) ¹	Střední až vysoká (závislá na kódování běžícím na pozadí prohlížeče)	Extrémní (100 % zatížení CPU softwarovým kódováním) ¹	Vysoká (přepočet pixelů a komprese ve WASM) ⁸

Technické zhodnocení limitů starších řešení

Zatímco rozhraní MediaRecorder API představuje jednoduchou cestu pro nahrávání plátna, pro produkční PWA aplikaci generující videoklipy je zcela nevhodné.³ Funguje totiž jako ekvivalent „nahrávání obrazovky běžícího programu“. ³ Pokud se procesor na okamžik zahltí vykreslováním komplexního vizuálního efektu, snímková frekvence výsledného videa klesne a dojde k

narušení synchronizace zvuku a obrazu.¹ Navíc toto rozhraní nutí uživatele čekat po celou dobu stopáže skladby (např. tříminutová skladba se nahrává přesně tři minuty).³

Integrace FFmpeg do WebAssembly se potýká s absencí hardwarové akcelerace pro kódování v prostředí WASM sandboxu.³ Proces komprese Full HD videa při 60 FPS čistě na CPU uvnitř prohlížeče generuje neúnosné teplotní zatížení a kódování delších skladeb často selhává kvůli překročení paměťového limitu 32bitového WASM adresního prostoru.³

Spojení WebGPU a WebCodecs představuje v roce 2026 dominantní technologický standard.¹ WebGPU zajišťuje bleskové vykreslení snímků nezávisle na obnovovací frekvenci monitoru a WebCodecs odesílá tyto snímky přímo do hardwarového kodéru grafické karty, což zkracuje čas generování tříminutového Full HD videoklipu na jednotky sekund.¹

Využití WebGPU pro generování audio-reaktivních vizuálů

Použití rozhraní WebGPU přináší zásadní výhody oproti staršímu standardu WebGL 2.¹ WebGPU je navrženo tak, aby reflektovalo architekturu moderních grafických čipů, což dramaticky snižuje režii CPU při volání vykreslovacích příkazů (draw calls).⁴ To umožňuje vykreslovat vysoce komplexní scény s desítkami tisíc nezávislých prvků.⁴

Výpočetní versus vykreslovací shadery

Klíčovým architektonickým rozdílem ve WebGPU je nativní podpora pro výpočetní průchody (compute passes) pomocí výpočetních shaderů (compute shaders) napsaných v jazyce WGSL (WebGPU Shading Language).¹ V tradičním WebGL musel být jakýkoliv vizuální efekt vyjádřen jako geometrický problém (vykreslení texturovaného obdélníku přes celou obrazovku).¹⁴

WebGPU umožňuje provádět čistě matematické simulace přímo v paměti GPU¹:

- **Výpočetní fáze (Compute Pass):** V této fázi běží shader, který počítá pozice částic, simuluje fyzikální interakce na základě audio dat, nebo provádí pokročilé obrazové efekty (např. rozostření, simulace tekutin, zpětná vazba).¹² Tento shader zapisuje výsledky do úložných bufferů (storage buffers) nebo textur.¹³
- **Vykreslovací fáze (Render Pass):** Následný vertex a fragment shader vezme data z úložných bufferů a vykreslí je na obrazovku (případně do textury plátna pro kódér).¹³

Díky této architektuře mohou audio-reaktivní částicové systémy čítat miliony částic běžících stabilně při 60 snímcích za sekundu, což by s využitím procesoru (CPU) nebo staršího WebGL nebylo klientsky realizovatelné.¹

Integrace s vyššími knihovnami a TSL

Ačkoli lze psát čistý WGSL kód, pro vývoj PWA aplikace je efektivnější využít vyšší grafické knihovny, které v roce 2026 disponují plnou podporou WebGPU.⁴ Knihovna **Three.js** (od verze r171) obsahuje kompletní WebGPURenderer s automatickým návratem k WebGL 2 (fallback) pro starší zařízení.⁴

Three.js pro WebGPU zavádí revoluční meta-jazyk **TSL (Three Shading Language)**.¹² TSL

umožňuje vývojářům psát shadery přímo v TypeScriptu pomocí uzlových struktur.¹² Knihovna následně tento kód automaticky překládá do optimalizovaného WGSL.¹² Pro komplexní 3D audio-reaktivní scény lze s úspěchem využít rovněž herní engine **BabylonJS** nebo **PlayCanvas**, které v roce 2026 kompletně přepsaly svá jádra do WGSL.⁴

Praktické ukázky a existující projekty

Při vývoji vlastní PWA aplikace lze čerpat inspiraci z několika úspěšných open-source projektů, které demonstrují možnosti klientského zpracování médií:

- **KubeezCut:** Profesionální multi-trackový nelineární video editor (NLE) běžící stoprocentně v prohlížeči.² Všechny vizuální efekty a transformace běží jako WebGPU shadery bez jakéhokoliv zatížení CPU během náhledu.² Export je realizován přes WebCodecs a podporuje formáty MP4, WebM, MOV a MKV s kodeky H.264, H.265, VP9 a AV1.²
- **Wav2Bar:** Otevřený software pro generování audio vizualizací, který umožňuje importovat audio stopy, vytvářet spektrální vizualizátory, částicové toky a exportovat výsledné video do formátu MP4 lokálně na grafické kartě uživatele.¹⁷
- **Audio Shader Studio:** Platforma pro real-time audio-reaktivní vizualizace drivené fragment shadery.¹⁸ Obsahuje pokročilou spektrální analýzu a mapuje audio funkce přímo na uniformní proměnné shaderu.¹⁸

Offline audio analýza a synchronizace snímků

Pro generování přesného audio-reaktivního videa, které dokonale lícuje s dynamikou skladby ze Suno AI, je nutné opustit model real-time analýzy, která se používá pro interaktivní přehrávače.¹ Při renderování videa mimo reálný čas (např. rychlostí 200 snímků za sekundu) by standardní AnalyserNode připojený k běžícímu audiu vrátil nesmyslná data, protože rychlost vykreslování neodpovídá rychlosti přehrávání zvuku.¹

Mechanismus OfflineAudioContext

Řešením je provedení kompletní analýzy zvukového souboru před zahájením samotného exportu videa.¹⁰ K tomu se využívá **OfflineAudioContext**, což je specializovaná verze audio kontextu určená pro rychlé zpracování zvuku na pozadí, běžící v samostatném systémovém vlákně.¹⁹

Postup analýzy a synchronizace probíhá v následujících krocích:

1. Uživatel nahraje audio stopu (např. MP3 export ze Suno AI) do PWA aplikace.¹⁰
2. Pomocí `AudioContext.decodeAudioData()` dojde k dekodování komprimovaného souboru do syrového objektu `AudioBuffer`.¹⁰
3. Určí se cílová snímková frekvence videa (např. $FPS = 30$ nebo $FPS = 60$).⁷
4. Celkový počet snímků videa se spočítá jako $Frames = duration \times FPS$.⁷
5. Pro každý snímek videa i (kde $i \in \langle 0, Frames - 1 \rangle$) se vypočítá přesné časové

razítka $t_i = i / FPS$ v sekundách.⁷

6. Vypočítá se odpovídající index vzorku v audio poli: $Index = t_i \times sampleRate$ ¹⁰

7. Pro toto časové okno se z dat levého a pravého kanálu AudioBuffer extrahují spektrální a dynamické charakteristiky.²⁶

Matematické základy spektrální analýzy

Získaná data z časového okna o velikosti N vzorků (typicky $N = 1024$ nebo $N = 2048$ vzorků, což odpovídá FFT velikosti) jsou podrobena matematické analýze.²⁶ Mezi nejčastěji extrahované parametry, které se následně předávají do WebGL shaderů jako uniformní proměnné (uniforms), patří RMS energie a spektrální těžiště ¹⁸:

RMS (Root Mean Square) Energie: Reprezentuje okamžitou hlasitost a fyzikální energii signálu v daném časovém úseku. Vypočítá se jako kvadratický průměr amplitud vzorků v časové doméně ¹⁸:

$$X_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x_n^2}$$

Kde x_n je amplituda n -tého vzorku v daném okně.¹⁸ Tato hodnota se v shaderech typicky mapuje na globální měřítko (např. pulzování objektů do rytmu basů).¹⁰

Spektrální těžiště (Spectral Centroid): Indikuje „střed hmoty“ frekvenčního spektra a koreluje s vnímáním jasu zvuku (např. zda v daný moment dominuje temný bas Suno AI syntetizátoru, nebo ostrý činel).¹⁸ Vypočítá se z frekvenčního spektra získaného pomocí rychlé Fourierovy transformace (FFT) podle vztahu ¹⁸:

$$C = \frac{\sum_{k=0}^{M-1} f_k \cdot |X_k|}{\sum_{k=0}^{M-1} |X_k|}$$

Kde $M = N/2$ je počet frekvenčních pásem (bins), f_k je střední frekvence k -tého pásma a $|X_k|$ je jeho magnituda (amplituda).¹⁸ Spektrální těžiště umožňuje shaderu dynamicky měnit barvy nebo aktivovat specifické grafické filtry pro vysoké frekvence.¹⁸

Integrace WebCodecs a multiplexování médií

Jakmile jsou audio charakteristiky extrahovány a WebGL scéna je připravena k vykreslování,

nastupuje proces kódování a multiplexování (muxování).³ WebCodecs API poskytuje nízkourovňové rozhraní pro kompresi médií, avšak samo o sobě neumí zapsat výsledný videosoubor.⁵ Výstupem z kodéru VideoEncoder jsou pouze komprimované binární kusy dat (EncodedVideoChunk), které je nutné poskládat do standardního kontejneru (MP4 nebo WebM) pomocí multiplexeru.⁵

Konverze AudioBuffer na f32-planar AudioData

Zatímco zpracování videa z plátna do VideoFrame je díky přímému propojení s GPU texturami triviální, zpracování zvuku vyžaduje specifickou datovou transformaci.¹ YouTube vyžaduje, aby výsledné MP4 video obsahovalo zvukovou stopu komprimovanou kodekem AAC.⁵

Nativní rozhraní WebCodecs AudioEncoder vyžaduje na vstupu objekty typu AudioData.⁵ Tyto objekty však nelze vytvořit přímo z rozhraní Web Audio API (AudioBuffer), protože tyto dvě specifikace spolu nativně nekomunikují.³¹ Vývojář musí ručně převést vícekanalová prokládaná data z AudioBuffer do lineárního jednorozměrného pole typu Float32Array ve formátu f32-planar (kde jsou vzorky pro jednotlivé kanály uloženy sekvenčně za sebou, nikoliv střídavě).²³

Matematicky, pokud má AudioBuffer dva kanály ($C = 2$) o délce L vzorků, výsledný Float32Array musí mít velikost $2 \times L$.²³ Prvních L pozic v poli obsahuje data levého kanálu, dalších L pozic data pravého kanálu.²³

TypeScript

```
function bufferToF32Planar(input: AudioBuffer): Float32Array
```

```
  const
```

```
  const
```

```
  const result = new Float32Array
```

```
  let offset = 0
```

```
  for let i = 0
```

```
    const
```

```
    result.set(channelData, offset)
```

```
    offset += length
```

```
  return result
```

Tento transformovaný buffer je následně předán konstruktoru `AudioData`, který jej zapouzdří pro nízkouúrovňový kódér.²³

Referenční implementace exportního řetězce

Následující TypeScript kód představuje kompletní, produkčně připravenou implementaci exportního řetězce, která využívá nízkouúrovňové rozhraní `WebCodecs` a knihovnu `mp4-muxer` (od autora Vanilagy) pro vytvoření videosouboru MP4 (H.264 / AAC) přímo v prohlížeči.⁷

TypeScript

```
import { Muxer } from "mp4-muxer"

interface GeneratorOptions {
  audioFile: File
  canvas: HTMLCanvasElement | OffscreenCanvas
  width: number
  height: number
  fps: number
  bitrate?: number
}

export async function exportVideoClip(options: GeneratorOptions): Promise<Blob> {
  const { audioFile, canvas, width, height, fps } = options
  const videoBitrate = options.bitrate || 5_000_000 // 5 Mbps pro 1080p YouTube standard

  // 1. Dekódování audio souboru ze Suno AI
  const audioCtx = new AudioContext({ sampleRate: 44100 })
  const arrayBuffer = await audioFile.arrayBuffer()
  const audioBuffer = await audioCtx.decodeAudioData(arrayBuffer)

  const totalFrames = Math.floor(audioBuffer.duration * fps)
  const sampleRate = audioBuffer.sampleRate
  const channels = audioBuffer.numberOfChannels

  // 2. Inicializace MP4 multiplexeru s podporou videa i audia
  const muxer = new Muxer({
    target: new Muxer.ArrayBufferTarget(),
    video
  })
```

```

    codec "avc" // H.264 standard [5, 7]
    width width
    height height
  }

  audio
    codec "aac" // Nativní audio kodek pro MP4/YouTube [5, 30, 34]
    numberOfChannels channels
    sampleRate sampleRate
  }

  fastStart "in-memory" // Optimalizace pro okamžité přehrávání [7]
}

```

```

// 3. Konfigurace nízkoúrovňového video kodéru (WebCodecs)
const videoEncoder = new VideoEncoder()
output (chunk, meta) => muxer.addVideoChunk(chunk, meta) 7
error (e) => console.error("Chyba video kodéru:" e)
}

```

```

videoEncoder.configure({
  codec "avc1.42001f" // Široce kompatibilní H.264 profil [7]
  width width
  height height
  bitrate videoBitrate
  bitrateMode "constant" 7
  avc format "avc" // Délkově prefixovaný formát pro MP4 multiplexery [24]
})

```

```

// 4. Konfigurace nízkoúrovňového audio kodéru
const audioEncoder = new AudioEncoder()
output (chunk, meta) => muxer.addAudioChunk(chunk, meta) 30
error (e) => console.error("Chyba audio kodéru:" e)
}

```

```

audioEncoder.configure({
  codec "mp4a.40.2" // AAC-LC profil vyžadovaný pro MP4 [30]
  numberOfChannels channels
  sampleRate sampleRate
  bitrate 128_000 // 128 kbps standard [23, 30]
})

```

```

// 5. Zakódování audio stopy
// Převod AudioBufferu na jednorozměrný f32-planar Float32Array
const planarAudioData = new Float32Array(audioBuffer.length * channels) 23

```

```

let audioOffset = 0
for let c = 0; c < channels; c++)
    planarAudioData.set(audioBuffer.getChannelData(c), audioOffset);
    audioOffset += audioBuffer.length;

const audioData = new AudioData(
    format "f32-planar" 23 32
    sampleRate sampleRate
    numberOfChannels channels
    numberOfFrames audioBuffer.length
    timestamp 0 // Audio začíná od času 0
    data planarAudioData
);

audioEncoder.encode(audioData) 23
audioData.close() // Okamžité uvolnění paměti [30]

// 6. Vykreslovací a kódovací smyčka videa (mimo reálný čas)
for let frameNum = 0; frameNum < totalFrames; frameNum++)
    // Výpočet přesného časového razítka v mikrosekundách (1e6 = 1 000 000)
    const timestampUs = (frameNum * 1e6 / fps) 7

    // TODO: V tomto místě vývojář provede aktualizaci své WebGPU scény
    // (např. předá RMS energii pro čas (timestampUs / 1e6) do shaderů a vyrenderuje snímek) [1,
18]

    // Vytvoření VideoFrame z plátna (přímé mapování GPU paměti)
    const frame = new VideoFrame(canvas, { timestamp timestampUs }); 6 7

    // Odeslání snímku do hardwarového kodéru
    videoEncoder.encode(frame) 7
    frame.close() // Kritické pro zamezení úniku paměti GPU

    // Zamezení zahlcení fronty kodéru (zpětný tlak)
    if (videoEncoder.encodeQueueSize > 15) 6 35
        await new Promise (resolve) =>
            videoEncoder.addEventListener "dequeue" resolve { once true } 35
    }

// 7. Dokončení zápisu a uzavření souboru

```

```

await videoEncoder.flush() 5 7
await audioEncoder.flush() 34 35

videoEncoder.close()
audioEncoder.close()

muxer.finalize() 7 34

// Výstupem je binární Blob reprezentující výsledné MP4 video
const buffer = muxer.target as Mp4Muxer.ArrayBufferTarget 7
return new Blob([buffer], {type: "video/mp4"}) 7 28

```

Tento kód demonstruje, jak lze celou video i audio pipeline zpracovat klientsky v jednom plynulém průchodu bez nutnosti jakékoliv serverové infrastruktury.¹⁰ Výsledný soubor MP4 s kodeky H.264 a AAC splňuje veškeré specifikace YouTube pro okamžité nahrání na platformu.⁵

Produkční výzvy a optimalizace v prostředí PWA

Při nasazení audio-reaktivního generátoru videoklipů jako progresivní webové aplikace (PWA) se vývojář setká s několika technickými specifikami prohlížečů, které vyžadují pokročilé optimalizační vzorce.²

Správa paměti a eliminace úniků (Memory Leaks)

Zpracování videa s vysokým rozlišením (např. 1080p při 60 FPS) vyžaduje obrovské množství datového toku.¹ Snímek v rozlišení 1920×1080 v nekomprimovaném formátu RGBA zabírá v paměti přibližně 8,3 MB. Při 60 FPS to představuje téměř 500 MB dat za jedinou sekundu videa.¹

Pokud vývojář zapomene volat metodu `.close()` na instancích `VideoFrame` a `AudioData`, prohlížeč neuvolní alokovanou grafickou paměť, což během několika sekund generování povede k pádu celé záložky prohlížeče (často s chybovým kódem signalizujícím vyčerpání paměti GPU).¹ Uvolňování musí probíhat synchronně a okamžitě po volání metody `.encode()`.¹

Optimalizace diskových operací pomocí OPFS

Ukládání výsledného videa do standardního vnitropaměťového bufferu (např. `ArrayBufferTarget` v `mp4-muxer`) naráží na limity RAM u mobilních zařízení a starších počítačů, kde může prohlížeč aplikaci ukončit při pokusu o alokaci souboru většího než 1–2 GB.⁸

V prostředí PWA je proto vysoce doporučeno nahradit vnitropaměťový zápis za **Origin Private File System (OPFS)**.² OPFS představuje vysoce výkonné, privátní souborové úložiště integrované přímo v prohlížeči, které umožňuje PWA aplikaci zapisovat data na disk uživatele extrémně rychle, asynchronně a s minimální režii.² Spojením OPFS s rozhraním `FileSystemWritableFileStream` lze zapisovat výsledný videosoubor přímo na disk průběžně

během kódování, což eliminuje jakékoliv paměťové limity a umožňuje exportovat i dlouhé videosety v rozlišení 4K.²

Bezpečnostní a platformové restrikce

Při klientské implementaci je nutné vyřešit několik bezpečnostních a ekosystémových specifik:

- **Zásada CORS (Cross-Origin Resource Sharing):** Pokud by aplikace načítala hudební soubory z externích cloudových úložišť (mimo doménu, na které běží PWA), prohlížeč z bezpečnostních důvodů zablokuje možnost analyzovat zvuk nebo exportovat textury z plátna.¹ Veškeré externí zdroje musí mít správně nakonfigurované CORS hlavičky, nebo musí být nahrány lokálně uživatelem přes prvek `<input type="file">`.¹
- **Politika přehrávání audia (Autoplay Policy):** Moderní prohlížeče neumožňují inicializovat `AudioContext` nebo spustit přehrávání zvuku bez předchozí explicitní interakce uživatele (např. kliknutí na tlačítko).²⁷ PWA aplikace proto musí obsahovat úvodní obrazovku s tlačítkem pro spuštění generování.²⁷
- **Běh na pozadí a Web Workers:** Pokud uživatel během exportu videa přepne záložku prohlížeče na jinou stránku, prohlížeč automaticky omezí výkon (throttling) pro neaktivní záložku, což by generování drasticky zpomalilo nebo zcela zastavilo.⁶ Pro zajištění plynulého exportu na pozadí je nezbytné přesunout celou renderovací a kódovací pipeline (včetně `OffscreenCanvas`, `WebGPU` kontextu a `WebCodecs`) do samostatného vlákna **Web Worker**.⁶

Závěr a doporučený technologický stack

Pro vývoj vlastní PWA aplikace pro generování audio-reaktivních videoklipů k hudbě ze Suno AI se jako nejefektivnější jeví následující technologický stack:

1. **Aplikační vrstva (PWA):** TypeScript, Vite (pro rychlý vývojový server a optimalizaci sestavení) a Service Workers pro offline běh aplikace.²
2. **Grafické jádro: Three.js** (verze r171+) využívající `WebGPURenderer` a shaderový meta-jazyk **TSL**.⁴ TSL dramaticky usnadňuje psaní komplexních reaktivních částicových simulací a přechodových efektů bez nutnosti psát nízkourovňový WGSL kód.¹²
3. **Audio analýza: OfflineAudioContext** pro rychlou, synchronní analýzu zvukové stopy na pozadí.¹⁹ Získané hodnoty RMS energie a spektrálního těžiště se namapují na časovou osu a předávají jako uniformní proměnné do Three.js shaderů v každém snímku.¹⁸
4. **Zpracování a komprese médií: Mediabunny** (případně nízkourovňová kombinace `WebCodecs` a knihovny `mp4-muxer`).⁷ Mediabunny představuje nejmodernější řešení pro klientské operace s médii v roce 2026, které zcela nahrazuje starší a nepodporované knihovny.⁹ Poskytuje elegantní API pro automatické kódování videa z plátna a současně bezpečně integruje audio stopu bez nutnosti ručního řešení složitých asynchronních front kodérů.⁹
5. **Ukládání souborů:** Kombinace **File System Access API** a **OPFS** pro průběžný zápis kódovaných dat na disk, což zaručuje stabilitu aplikace i při generování velkých video souborů pro YouTube.²

Citovaná díla

1. Video editing in the browser with WebGPU | Joan León, použito května 28, 2026, <https://joanleon.dev/en/webgpu-video-browser/>
2. I Built a Free Browser Video Editor with WebGPU, WebCodecs & Optional AI Generation, použito května 28, 2026, <https://dev.to/sebyx07/i-built-a-free-browser-video-editor-with-webgpu-webcodecs-optional-ai-generation-2eo0>
3. The WebCodecs Handbook: Native Video Processing in the Browser - freeCodeCamp, použito května 28, 2026, <https://www.freecodecamp.org/news/the-webcodecs-handbook-native-video-processing-in-the-browser/>
4. WebGPU 2026: 70% Browser Support, 15x Performance Gains ..., použito května 28, 2026, <https://byteiota.com/webgpu-2026-70-browser-support-15x-performance-gains/>
5. WebCodecs API - MDN Web Docs, použito května 28, 2026, https://developer.mozilla.org/en-US/docs/Web/API/WebCodecs_API
6. Video processing with WebCodecs | Web Platform - Chrome for Developers, použito května 28, 2026, <https://developer.chrome.com/docs/web-platform/best-practices/webcodecs>
7. How to Save HTML Canvas to Mp4 Using WebCodecs API 10x ..., použito května 28, 2026, <https://devtails.xyz/adam/how-to-save-html-canvas-to-mp4-using-web-codecs-api>
8. dmnsn/canvas-record: Record a video in the browser or ... - GitHub, použito května 28, 2026, <https://github.com/dmnsn/canvas-record>
9. Introduction - Mediabunny - Mintlify, použito května 28, 2026, <https://www.mintlify.com/explore/Vanilagymediabunny>
10. Free Online Audio Visualizer Video Maker - AudioEditor, použito května 28, 2026, <https://audioeditor.org/tools/audio-visualizer-video>
11. Video & Audio Muxer - WebBrowserTools.com, použito května 28, 2026, <https://webbrowsertools.com/video-audio-muxer/>
12. Migrate Three.js to WebGPU (2026) — The Complete Checklist - Utsubo, použito května 28, 2026, <https://www.utsubo.com/blog/webgpu-threejs-migration-guide>
13. WebGPU Shading Language - W3C, použito května 28, 2026, <https://www.w3.org/TR/WGSL/>
14. Why WebGPU Feels Like the Future of the Web (Live Demo) - DEV Community, použito května 28, 2026, <https://dev.to/sylwia-lask/why-webgpu-feels-like-the-future-of-the-web-live-demo--2bjh>
15. WebGPU Hits Critical Mass: All Major Browsers Now Ship It, použito května 28, 2026, <https://www.webgpu.com/news/webgpu-hits-critical-mass-all-major-browsers/>
16. Shader Editor - with WebGPU compute and AudioWorklet sound - Reddit, použito května 28, 2026,

- https://www.reddit.com/r/webgpu/comments/1okal12/shader_editor_with_webgpu_compute_and/
17. Wav2Bar - Free, open source audio visualization software - Picorims, použito května 28, 2026, <https://picorims.github.io/wav2bar-website/>
 18. sandner-art/Audio-Shader-Studio: Audio reactive shader app for experiments. WebGL for high-performance shader rendering and the Web Audio API for real-time audio analysis. - GitHub, použito května 28, 2026, <https://github.com/sandner-art/Audio-Shader-Studio>
 19. Using the Web Audio API - MDN Web Docs, použito května 28, 2026, https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Using_Web_Audio_API
 20. webaudio-examples | Code examples that accompany the MDN Web Docs pages relating to Web Audio. - GitHub Pages, použito května 28, 2026, <https://mdn.github.io/webaudio-examples/>
 21. how can i use web audio api with offscreen canvas? - Stack Overflow, použito května 28, 2026, <https://stackoverflow.com/questions/67967751/how-can-i-use-web-audio-api-with-offscreen-canvas>
 22. I made a browser-based tool to create audio-reactive visuals for your beats (Free / 4K Export) - Reddit, použito května 28, 2026, https://www.reddit.com/r/LofiHipHop/comments/1qj2ka6/i_made_a_browserbased_tool_to_create/
 23. Encoding an AudioBuffer with web codecs - Stack Overflow, použito května 28, 2026, <https://stackoverflow.com/questions/78074618/encoding-an-audiobuffer-with-web-codecs>
 24. create a video file encoded using WebCodecs · Issue #243 · gpac/mp4box.js - GitHub, použito května 28, 2026, <https://github.com/gpac/mp4box.js/issues/243>
 25. Mediabunny - ffmpeg for the web | WebCodecs Fundamentals, použito května 28, 2026, <https://webcodecsfundamentals.org/projects/media-bunny/>
 26. Visualizations with Web Audio API - MDN Web Docs, použito května 28, 2026, https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API/Visualizations_with_Web_Audio_API
 27. Part 1: Building a Real-Time Audio Visualizer with Web Audio API and Canvas | by Senthil Kumaran Chinnathambi | IceApple DevLogs, použito května 28, 2026, <https://medium.com/iceapple-tech-talks/web-audio-api-part-1-playing-filtering-and-visualizing-sound-147254dd6379>
 28. Muxing and Demuxing - WebCodecs Fundamentals, použito května 28, 2026, <https://webcodecsfundamentals.org/basics/muxing/>
 29. Using the WebCodecs API - MDN Web Docs, použito května 28, 2026, https://developer.mozilla.org/en-US/docs/Web/API/WebCodecs_API/Using_the_WebCodecs_API
 30. How to encode audio buffer for mp4-muxer using only client side? - Stack Overflow, použito května 28, 2026, <https://stackoverflow.com/questions/78431537/how-to-encode-audio-buffer-for-mp>

[4-muxer-using-only-client-side](#)

31. Intro to Audio - WebCodecs Fundamentals, použito května 28, 2026, <https://webcodecsfundamentals.org/audio/intro/>
32. AudioData() constructor - Web APIs - MDN Web Docs - Mozilla, použito května 28, 2026, <https://developer.mozilla.org/en-US/docs/Web/API/AudioData/AudioData>
33. mp4-muxer - A CDN for npm and GitHub - jsDelivr, použito května 28, 2026, <https://www.jsdelivr.com/package/npm/mp4-muxer>
34. Has anyone built a web-based video editor that uses WebGPU for the compositing/rendering layer? - Reddit, použito května 28, 2026, https://www.reddit.com/r/webgpu/comments/1rivz9e/has_anyone_built_a_webbased_video_editor_that/
35. WebCodecs API Samples - W3C on GitHub, použito května 28, 2026, <https://w3c.github.io/webcodecs/samples/>
36. Write an audio visualizer from scratch with vanilla JavaScript - LogRocket Blog, použito května 28, 2026, <https://blog.logrocket.com/audio-visualizer-from-scratch-javascript/>
37. Mediabunny — A complete JavaScript media toolkit for the browser, použito května 28, 2026, <https://mediabunny.dev/>
38. Vanilagy/webm-muxer: WebM multiplexer in pure TypeScript with support for WebCodecs API, video & audio. - GitHub, použito května 28, 2026, <https://github.com/Vanilagy/webm-muxer>
39. GitHub - Vanilagy/mp4-muxer: MP4 multiplexer in pure TypeScript with support for WebCodecs API, video & audio., použito května 28, 2026, <https://github.com/Vanilagy/mp4-muxer>
40. Introduction | Mediabunny, použito května 28, 2026, <https://mediabunny.dev/guide/introduction>
41. Vanilagy/mediabunny: Pure TypeScript media toolkit for ... - GitHub, použito května 28, 2026, <https://github.com/Vanilagy/mediabunny>